# Realice puestas en marcha con el operador de Trident

Astra Trident

NetApp
November 20, 2023

This PDF was generated from https://docs.netapp.com/es-es/trident-2110/trident-get-started/kubernetes-customize-deploy.html on November 20, 2023. Always check docs.netapp.com for the latest.

# Tabla de contenidos

Realice puestas en marcha con el operador de Trident . . . 1
Ponga en marcha el operador de Trident mediante Helm . . . 1
Ponga en marcha manualmente el operador de Trident . . . 2
Personalice la implementación del operador de Trident . . . 7

# Realice puestas en marcha con el operador de Trident

Puede poner en marcha Astra Trident con el operador de Trident. Es posible poner en marcha el operador de Trident de forma manual o mediante Helm.

> Si usted no se ha familiarizado ya con el "conceptos básicos", ahora es un gran momento para hacerlo.

**Lo que necesitará**

Para poner en marcha Astra Trident, se deben cumplir los siguientes requisitos previos:

- Tiene privilegios completos para un clúster de Kubernetes compatible que ejecute Kubernetes 1.17 y versiones posteriores.
- Tiene acceso a un sistema de almacenamiento de NetApp compatible.
- Puede montar volúmenes de todos los nodos de trabajo de Kubernetes.
- Tiene un host Linux con `kubectl` (o. `oc`, Si está utilizando OpenShift) instalado y configurado para administrar el clúster de Kubernetes que desea utilizar.
- Ha configurado el `KUBECONFIG` Variable de entorno para señalar la configuración del clúster de Kubernetes.
- Habilitó el "Puertas de funciones requeridas por Astra Trident".
- Si utiliza Kubernetes con Docker Enterprise, "Siga sus pasos para habilitar el acceso a la CLI".

¿Tiene todo eso? Estupendo. Empecemos:

## Ponga en marcha el operador de Trident mediante Helm

Realice los pasos que se enumeran para implementar el operador de Trident mediante Helm.

**Lo que necesitará**

Además de los requisitos previos mencionados anteriormente, para poner en marcha el operador de Trident con Helm, es necesario lo siguiente:

- Kubernetes 1.17 y posteriores
- Versión timón 3

**Pasos**

1. Descargue el paquete del instalador de desde la "Trident GitHub" página. El paquete de instalación incluye el gráfico Helm en la `/helm` directorio.
2. Utilice la `helm install` y especifique un nombre para la implementación. Consulte el siguiente ejemplo:

   ```
   helm install <name> trident-operator-21.07.1.tgz --namespace <namespace
   you want to use for Trident>
   ```

   Si todavía no creó un espacio de nombres para Trident, puede añadir el `--create-namespace`

parámetro de la `helm install` comando. Helm creará automáticamente el espacio de nombres para usted.

Existen dos formas de pasar los datos de configuración durante la instalación:

- `--values` (o. `-f`): Especifique un archivo YAML con anulaciones. Esto se puede especificar varias veces y el archivo de la derecha tendrá prioridad.
- `--set`: Especifique anulaciones en la línea de comandos.

Por ejemplo, para cambiar el valor predeterminado de `debug`, ejecute lo siguiente `--set` comando:

```
$ helm install <name> trident-operator-21.07.1.tgz --set tridentDebug=true
```

La `values.yaml` Archivo, que forma parte del gráfico Helm, proporciona la lista de claves y sus valores predeterminados.

`helm list` muestra detalles sobre la instalación, como nombre, espacio de nombres, gráfico, estado, versión de la aplicación, número de revisión, etc.

# Ponga en marcha manualmente el operador de Trident

Realice los pasos que se enumeran para implementar manualmente el operador de Trident.

## Paso 1: Califique su clúster de Kubernetes

Lo primero que debe hacer es iniciar sesión en el host Linux y comprobar que está gestionando un *working*, "Clúster de Kubernetes compatible" que tenga los privilegios necesarios para.

> Con OpenShift, utilícelo `oc` en lugar de `kubectl` en todos los ejemplos que siguen, e inicie sesión como **system:admin** primero ejecutando `oc login -u system:admin` o. `oc login -u kube-admin`.

Para ver si la versión de Kubernetes es posterior a la 1.17, ejecute el siguiente comando:

```
kubectl version
```

Para ver si tiene privilegios de administrador de clúster Kubernetes, ejecute el siguiente comando:

```
kubectl auth can-i '*' '*' --all-namespaces
```

Para verificar si puede iniciar un pod que utiliza una imagen desde Docker Hub y llegar al sistema de almacenamiento a través de la red de pod, ejecute el siguiente comando:

```
kubectl run -i --tty ping --image=busybox --restart=Never --rm -- \
  ping <management IP>
```

## Paso 2: Descargue y configure el operador

> A partir de 21.01, el operador de Trident se limita al clúster. Para usar el operador de Trident para la instalación de Trident, se debe crear el `TridentOrchestrator` Definición de recursos personalizados (CRD) y definición de otros recursos. Debe realizar estos pasos para configurar el operador antes de poder instalar Astra Trident.

1. Descargue la última versión de "Paquete de instalación de Trident" De la sección *Downloads* y extráigalo.

```
wget https://github.com/NetApp/trident/releases/download/v21.04/trident-
installer-21.04.tar.gz
tar -xf trident-installer-21.04.tar.gz
cd trident-installer
```

2. Utilice el manifiesto CRD adecuado para crear `TridentOrchestrator` CRD. A continuación, cree un `TridentOrchestrator` Recursos personalizados más adelante para crear una instancia de la instalación por parte del operador.

   Ejecute el siguiente comando:

```
kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
```

3. Después del `TridentOrchestrator` Cree CRD, cree los siguientes recursos necesarios para la implementación del operador:

   - Una cuenta de servicio para el operador
   - Una función de clúster y ClusterRoleBinding a la cuenta de servicio
   - Una política de seguridad dedicada
   - El propio operador

   El instalador de Trident contiene manifiestos para definir estos recursos. De forma predeterminada, el operador se implementa en la `trident` espacio de nombres. Si la `trident` el espacio de nombres no existe; utilice el manifiesto siguiente para crear uno.

```
$ kubectl apply -f deploy/namespace.yaml
```

4. Para desplegar el operador en un espacio de nombres distinto del predeterminado `trident` namespace, debe actualizar el `serviceaccount.yaml`, `clusterrolebinding.yaml` y.. `operator.yaml` manifiesta y genera tu `bundle.yaml`.

   Ejecute el siguiente comando para actualizar los manifiestos de YAML y generar el `bundle.yaml` con el `kustomization.yaml`:

```
kubectl kustomize deploy/ > deploy/bundle.yaml
```

Ejecute el comando siguiente para crear los recursos e implementar el operador:

```
kubectl create -f deploy/bundle.yaml
```

5. Para verificar el estado del operador después de la implementación, haga lo siguiente:

```
$ kubectl get deployment -n <operator-namespace>
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
trident-operator   1/1     1            1           3m

$ kubectl get pods -n <operator-namespace>
NAME                                READY   STATUS             RESTARTS
AGE
trident-operator-54cb664d-lnjxh   1/1     Running            0
3m
```

La implementación del operador crea correctamente un pod que se ejecuta en uno de los nodos de trabajo del clúster.

> Solo debe haber **una instancia** del operador en un clúster de Kubernetes. No cree varias implementaciones del operador Trident.

## Paso 3: Crear `TridentOrchestrator` E instale Trident

Ahora está listo para instalar Astra Trident con el operador. Esto requerirá crear `TridentOrchestrator`. El instalador de Trident incluye definiciones de ejemplo para su creación `TridentOrchestrator`. Esto inicia una instalación en `trident` espacio de nombres.

```
$ kubectl create -f deploy/crds/tridentorchestrator_cr.yaml
tridentorchestrator.trident.netapp.io/trident created

$ kubectl describe torc trident
Name:        trident
Namespace:
Labels:      <none>
Annotations: <none>
API Version: trident.netapp.io/v1
Kind:        TridentOrchestrator
...
Spec:
  Debug:     true
  Namespace: trident
Status:
  Current Installation Params:
    IPv6:                      false
    Autosupport Hostname:
    Autosupport Image:         netapp/trident-autosupport:21.04
    Autosupport Proxy:
    Autosupport Serial Number:
    Debug:                     true
    Enable Node Prep:          false
    Image Pull Secrets:
    Image Registry:
    k8sTimeout:           30
    Kubelet Dir:          /var/lib/kubelet
    Log Format:           text
    Silence Autosupport:  false
    Trident Image:        netapp/trident:21.04.0
  Message:                  Trident installed  Namespace:
trident
  Status:                   Installed
  Version:                  v21.04.0
Events:
    Type Reason Age From Message ---- ------ ---- ---- -------Normal
    Installing 74s trident-operator.netapp.io Installing Trident Normal
    Installed 67s trident-operator.netapp.io Trident installed
```

El operador Trident le permite personalizar la manera en que se instala Astra Trident mediante los atributos del `TridentOrchestrator` espec. Consulte "Personalice su implementación de Trident".

El estado de `TridentOrchestrator` Indica si la instalación se realizó correctamente y muestra la versión de Trident instalada.

| Estado | Descripción |
|---|---|
| Instalación | El operador está instalando Astra Trident con este método `TridentOrchestrator` CR. |
| Instalado | Astra Trident se ha instalado correctamente. |
| Desinstalando | El operador está desinstalando Astra Trident, porque `spec.uninstall=true`. |
| Desinstalado | Astra Trident se desinstala. |
| Error | El operador no pudo instalar, aplicar parches, actualizar o desinstalar Astra Trident; el operador intentará recuperarse automáticamente de este estado. Si este estado continúa, necesitará solucionar problemas. |
| Actualizando | El operador está actualizando una instalación existente. |
| Error | La `TridentOrchestrator` no se utiliza. Otro ya existe. |

Durante la instalación, el estado de `TridentOrchestrator` cambios de `Installing` para `Installed`. Si observa la `Failed` y el operador no puede recuperar por sí solo, debe comprobar los registros del operador. Consulte "resolución de problemas" sección.

Puede confirmar si la instalación de Astra Trident se ha completado examinando los pods que se han creado:

```
$ kubectl get pod -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-7d466bf5c7-v4cpw        5/5     Running   0           1m
trident-csi-mr6zc                   2/2     Running   0           1m
trident-csi-xrp7w                   2/2     Running   0           1m
trident-csi-zh2jt                   2/2     Running   0           1m
trident-operator-766f7b8658-ldzsv   1/1     Running   0           3m
```

También puede utilizar `tridentctl` Para comprobar la versión de Astra Trident instalada.

```
$ ./tridentctl -n trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.04.0        | 21.04.0        |
+----------------+----------------+
```

Ahora puede Adelante y crear un back-end. Consulte "tareas posteriores a la implementación".

Para obtener información sobre la solución de problemas durante la implementación, consulte "resolución de problemas" sección.

# Personalice la implementación del operador de Trident

El operador Trident le permite personalizar la manera en que se instala Astra Trident mediante los atributos del `TridentOrchestrator` espec.

Consulte la tabla siguiente para ver la lista de atributos:

| Parámetro | Descripción | Predeterminado |
|---|---|---|
| `namespace` | Espacio de nombres para instalar Astra Trident en | "predeterminado" |
| `debug` | Habilite la depuración para Astra Trident | falso |
| `IPv6` | Instale Astra Trident sobre IPv6 | falso |
| `k8sTimeout` | Tiempo de espera para las operaciones de Kubernetes | 30 seg |
| `silenceAutosupport` | No envíe paquetes AutoSupport a NetApp automáticamente | falso |
| `enableNodePrep` | Administrar automáticamente las dependencias del nodo de trabajo (**BETA**) | falso |
| `autosupportImage` | La imagen contenedora para telemetría AutoSupport | "netapp/trident-autosupport:21.04.0" |
| `autosupportProxy` | La dirección/puerto de un proxy para enviar telemetría AutoSupport | "<a href="http://proxy.example.com:8888"" class="bare">http://proxy.example.com:8888"</a> |
| `uninstall` | Una Marca utilizada para desinstalar Astra Trident | falso |
| `logFormat` | Formato de registro de Astra Trident para utilizar [text,json] | "texto" |
| `tridentImage` | Imagen de Astra Trident para instalar | "netapp/trident:21.04" |
| `imageRegistry` | Ruta de acceso al registro interno, del formato `<registry FQDN>[:port][/subpath]` | "k8s.gcr.io/sig-storage (k8s 1.17+) o quay.io/k8scsi" |
| `kubeletDir` | Ruta al directorio kubelet del host | “/var/lib/kubelet” |
| `wipeout` | Una lista de recursos para eliminar y realizar una eliminación completa de Astra Trident | |

| Parámetro | Descripción | Predeterminado |
| --- | --- | --- |
| `imagePullSecrets` | Secretos para extraer imágenes de un registro interno | |

> `spec.namespace` se especifica en `TridentOrchestrator` Para indicar en qué espacio de nombres está instalado Astra Trident. Este parámetro **no se puede actualizar después de instalar Astra Trident**. Intentar hacerlo provoca el estado de `TridentOrchestrator` para cambiar a. `Failed`. Astra Trident no pretende migrar entre espacios de nombres.

> La preparación automática del nodo de trabajo es una **función beta** que se utiliza únicamente en entornos no productivos.

Puede utilizar los atributos mencionados anteriormente al definir `TridentOrchestrator` para personalizar la instalación. Veamos un ejemplo:

```
$ cat deploy/crds/tridentorchestrator_cr_imagepullsecrets.yaml
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident
  tridentImage: netapp/trident:21.04.0
  imagePullSecrets:
  - thisisasecret
```

Si desea personalizar la instalación más allá de lo que desee `TridentOrchestrator` los argumentos permiten, debe considerar utilizar `tridentctl` Para generar manifiestos YAML personalizados que puede modificar según sea necesario.

**Información de copyright**

Copyright © 2023 NetApp, Inc. Todos los derechos reservados. Imprimido en EE. UU. No se puede reproducir este documento protegido por copyright ni parte del mismo de ninguna forma ni por ningún medio (gráfico, electrónico o mecánico, incluidas fotocopias, grabaciones o almacenamiento en un sistema de recuperación electrónico) sin la autorización previa y por escrito del propietario del copyright.

El software derivado del material de NetApp con copyright está sujeto a la siguiente licencia y exención de responsabilidad:

ESTE SOFTWARE LO PROPORCIONA NETAPP «TAL CUAL» Y SIN NINGUNA GARANTÍA EXPRESA O IMPLÍCITA, INCLUYENDO, SIN LIMITAR, LAS GARANTÍAS IMPLÍCITAS DE COMERCIALIZACIÓN O IDONEIDAD PARA UN FIN CONCRETO, CUYA RESPONSABILIDAD QUEDA EXIMIDA POR EL PRESENTE DOCUMENTO. EN NINGÚN CASO NETAPP SERÁ RESPONSABLE DE NINGÚN DAÑO DIRECTO, INDIRECTO, ESPECIAL, EJEMPLAR O RESULTANTE (INCLUYENDO, ENTRE OTROS, LA OBTENCIÓN DE BIENES O SERVICIOS SUSTITUTIVOS, PÉRDIDA DE USO, DE DATOS O DE BENEFICIOS, O INTERRUPCIÓN DE LA ACTIVIDAD EMPRESARIAL) CUALQUIERA SEA EL MODO EN EL QUE SE PRODUJERON Y LA TEORÍA DE RESPONSABILIDAD QUE SE APLIQUE, YA SEA EN CONTRATO, RESPONSABILIDAD OBJETIVA O AGRAVIO (INCLUIDA LA NEGLIGENCIA U OTRO TIPO), QUE SURJAN DE ALGÚN MODO DEL USO DE ESTE SOFTWARE, INCLUSO SI HUBIEREN SIDO ADVERTIDOS DE LA POSIBILIDAD DE TALES DAÑOS.

NetApp se reserva el derecho de modificar cualquiera de los productos aquí descritos en cualquier momento y sin aviso previo. NetApp no asume ningún tipo de responsabilidad que surja del uso de los productos aquí descritos, excepto aquello expresamente acordado por escrito por parte de NetApp. El uso o adquisición de este producto no lleva implícita ninguna licencia con derechos de patente, de marcas comerciales o cualquier otro derecho de propiedad intelectual de NetApp.

Es posible que el producto que se describe en este manual esté protegido por una o más patentes de EE. UU., patentes extranjeras o solicitudes pendientes.

LEYENDA DE DERECHOS LIMITADOS: el uso, la copia o la divulgación por parte del gobierno están sujetos a las restricciones establecidas en el subpárrafo (b)(3) de los derechos de datos técnicos y productos no comerciales de DFARS 252.227-7013 (FEB de 2014) y FAR 52.227-19 (DIC de 2007).

Los datos aquí contenidos pertenecen a un producto comercial o servicio comercial (como se define en FAR 2.101) y son propiedad de NetApp, Inc. Todos los datos técnicos y el software informático de NetApp que se proporcionan en este Acuerdo tienen una naturaleza comercial y se han desarrollado exclusivamente con fondos privados. El Gobierno de EE. UU. tiene una licencia limitada, irrevocable, no exclusiva, no transferible, no sublicenciable y de alcance mundial para utilizar los Datos en relación con el contrato del Gobierno de los Estados Unidos bajo el cual se proporcionaron los Datos. Excepto que aquí se disponga lo contrario, los Datos no se pueden utilizar, desvelar, reproducir, modificar, interpretar o mostrar sin la previa aprobación por escrito de NetApp, Inc. Los derechos de licencia del Gobierno de los Estados Unidos de América y su Departamento de Defensa se limitan a los derechos identificados en la cláusula 252.227-7015(b) de la sección DFARS (FEB de 2014).

**Información de la marca comercial**

NETAPP, el logotipo de NETAPP y las marcas que constan en http://www.netapp.com/TM son marcas comerciales de NetApp, Inc. El resto de nombres de empresa y de producto pueden ser marcas comerciales de sus respectivos propietarios.